CitizenSense — Final Report

AIRI Team PITB — AI Internship Task 4 (End-to-End NLP Project) Project: CitizenSense — NLP-Based Public Feedback Classification and Insight System **Option selected:** A — Public Complaint Classifier (with Sentiment as the required secondary output) **Mode:** Individual task

---

1. Project Overview

CitizenSense is an NLP prototype that takes a short piece of public feedback (a complaint, review, or support message) and automatically produces a structured "insight card": **Category**, **Sentiment**, **Priority**, **Keywords**, a short **Summary**, and a **Suggested Department** for routing. Organizations receiving large volumes of citizen feedback cannot manually triage every message; CitizenSense demonstrates how a lightweight NLP pipeline can turn unstructured text into consistent, structured output for faster decision-making. The system is trained and evaluated on a synthetic, template-generated dataset of 739 feedback messages across 8 complaint categories, and predicts Category via a trained baseline model and Sentiment via a pretrained Hugging Face transformer pipeline.

2. Tools and Technologies Used

Python, Pandas, NumPy, scikit-learn (TF-IDF + Logistic Regression), Hugging Face Transformers (`sentiment-analysis` pipeline, DistilBERT/SST-2), PyTorch (backend for the transformer), Matplotlib (confusion matrix), Streamlit (demo UI), FastAPI (REST demo), Jupyter Notebook.

3. Dataset Preparation

- **Source:** 100% synthetic — written by the intern as ~100 sentence templates covering 8 categories, combined with randomized area/duration placeholders for natural variation. No public dataset was used; this is stated explicitly per the task's data-source rule. No private personal information, phone numbers, CNIC numbers, or addresses are present anywhere in the data.
- **Size:** 739 samples after deduplication, across 8 categories (smallest class: Education, 72 samples; largest: Roads/Infrastructure, 128 samples).
- **Labels:** Category (8 classes, required), Sentiment (Negative/Neutral/Positive, the chosen secondary label), Priority (Low/Medium/High/Urgent, generated but not separately modeled — see Limitations).
- **Multilingual slice:** ~6% of samples are Roman Urdu / mixed English-Urdu (e.g. *"hamare area ki road bohat kharab hai aur pani jama hota hai"*), exercising basic multilingual robustness.
- **Deliberately ambiguous slice:** a further slice reuses the same vague wording (e.g. *"There is a problem in our area that needs attention."*) across multiple randomly assigned categories, so that classification difficulty — and therefore model errors — are genuine rather than synthetic datasets being artificially easy.
- **Cleaning:** lowercasing, URL/phone-number stripping, punctuation handling, whitespace normalization, exact-duplicate removal.
- **Split:** stratified 70% train (517) / 15% validation (111) / 15% test (111).

4. Methodology

1. Collected/generated text data with category, sentiment, and priority labels.
2. Cleaned the dataset and performed a stratified train/val/test split.
3. Converted cleaned text into TF-IDF features (unigrams + bigrams, 5000 max features).
4. Trained a Logistic Regression baseline (`class_weight="balanced"`) for category classification.
5. Used a Hugging Face pretrained `sentiment-analysis` pipeline for Sentiment, adding a confidence threshold to recover a Neutral class from the binary pipeline output.

6. Extracted keywords with a corpus-fit TF-IDF vectorizer (real document-frequency statistics) and generated a structured one-line summary per message.
7. Evaluated the baseline model (accuracy, precision, recall, F1, confusion matrix, per-class report) and the sentiment pipeline (zero-shot evaluation against the dataset's ground-truth sentiment labels).
8. Performed manual error analysis on 15 reviewed cases.
9. Built three interchangeable demo interfaces: CLI, Streamlit, FastAPI.

5. Results

Baseline category classifier (TF-IDF + Logistic Regression), test set (111 samples):

Metric	Value
Accuracy	0.928
Weighted Precision	0.933
Weighted Recall	0.928
Weighted F1	0.929

- **Best-performing class:** Electricity (F1 = 1.00).
- **Weakest classes:** General and Water Supply (F1 ≈ 0.86) — "General" is the catch-all category and most overlaps with the deliberately ambiguous examples.
- **Main confusions (from the confusion matrix):** Roads/Infrastructure ↔ Water Supply/Sanitation, Electricity ↔ General, Health ↔ Education.

Hugging Face sentiment pipeline (zero-shot, test set):

Metric	Value
Accuracy	0.748
Negative F1	0.85
Neutral F1	0.08
Positive F1	0.73

The pretrained sentiment model is strong on clearly Negative/Positive text but struggles badly on Neutral (informational/question) messages — a realistic limitation of using an off-the-shelf English sentiment model for civic-complaint text without fine-tuning.

Full artifacts: `outputs/classification_report.txt`, `outputs/confusion_matrix.png`, `outputs/sentiment_evaluation.txt`.

6. Demo and Outputs

Three interfaces share one analyzer (`app/inference.py`):

- **CLI** (`python app/demo.py`) — type feedback text, get a printed insight card.
- **Streamlit** (`streamlit run app/streamlit_app.py`) — text box, Predict button, metrics for Category/Sentiment/Priority, confidence bars, keywords, summary, suggested department, and a raw-JSON expander.

- **FastAPI** (`uvicorn app.api:app --app-dir app`) — `POST /analyze-feedback`.

Example (matching the task brief's own example input):

```
Input: "The road in our area has been broken for months and rainwater collects
every day. Please fix it urgently."

Category: Roads / Infrastructure (confidence 0.60)
Sentiment: Negative (confidence 0.88)
Priority: High
Keywords: road area, collects, collects day, day fix, fix urgently
Suggested Department: Municipal / Infrastructure Department
Summary: Citizen reports a roads / infrastructure issue: the road in our area has
been broken for months and rainwater collects every day. please fix it urgently.
```

Screenshots: see `screenshots/`.

7. Error Analysis

15 cases were manually reviewed (14 genuine misclassifications + 1 low-confidence correct prediction), from the combined validation+test predictions. Full table: `outputs/error_analysis.csv`.

Representative failures mirror the task brief's own example error table:

Text Sample	Actual	Predicted	Likely reason
"No action has been taken in the colony for a long time."	Roads/Infrastructure	Water Supply	Vague wording also plausible as a general/water complaint
"We have been facing this problem in the housing scheme since last month."	Water Supply	Electricity	Generic phrasing, no category-specific keyword
"Please resolve the issue in sector G-9 as soon as possible."	Traffic	Health	Short, generic text with no distinguishing keyword

Why these happen: most errors come from the deliberately ambiguous template slice, where even a human reader could not reliably infer the category from text alone — this is an honest reflection of real short citizen complaints, not a flaw unique to the model.

Suggested improvements: 1. Add more labeled examples for the weaker classes (General, Water Supply, Health). 2. Balance ambiguous/overlapping classes with more distinctive examples. 3. Improve label guidelines for short/vague complaints. 4. Expand Roman Urdu / multilingual training data. 5. Fine-tune a transformer (e.g. DistilBERT) to use context beyond bag-of-words. 6. Add a human-review queue for low-confidence predictions.

8. What I Learned

- How to design and generate a labeled text dataset from scratch, including how to deliberately inject realistic ambiguity rather than relying on an artificially "too easy" separable dataset.
- How to apply and justify specific text-cleaning/preprocessing steps for short complaint-style text (and when *not* to apply stopword removal/stemming).
- How TF-IDF represents text, and how fitting it on a full corpus versus a single document changes the quality of extracted keywords.

- How to train and evaluate a multi-class text classifier end-to-end with scikit-learn, including stratified splitting and per-class metrics.
- How to use a Hugging Face pretrained pipeline for a task it wasn't designed for out-of-the-box (recovering a 3-way sentiment label from a binary classifier via a confidence threshold) and how to honestly evaluate its limitations.
- How to perform structured error analysis and turn it into concrete next steps.
- How to package an NLP model behind multiple demo interfaces (CLI, Streamlit, API) sharing one inference module.

9. Future Improvements

- Add full Urdu script and more Roman Urdu support.
- Add multi-label classification for complaints spanning more than one category.
- Fine-tune a transformer (e.g. DistilBERT) on CitizenSense categories directly.
- Add a dedicated, trained Priority classifier rather than a keyword heuristic.
- Add dashboard charts (category distribution, trends over time).
- Add duplicate/near-duplicate complaint detection and semantic search.
- Deploy the FastAPI service and Streamlit demo publicly.

10. Limitations

- All data is synthetic/template-generated, not real citizen complaints.
- Priority at inference time uses a simple keyword+sentiment heuristic rather than a trained classifier.
- The sentiment pipeline is general-purpose English and not fine-tuned for civic complaints or Roman Urdu text (see Section 5 — Neutral F1 is weak).
- Categories are single-label; real complaints can span multiple issues at once.
- This is a learning prototype, not a production complaint-management system, and its predictions should be treated as suggestions, not final decisions.